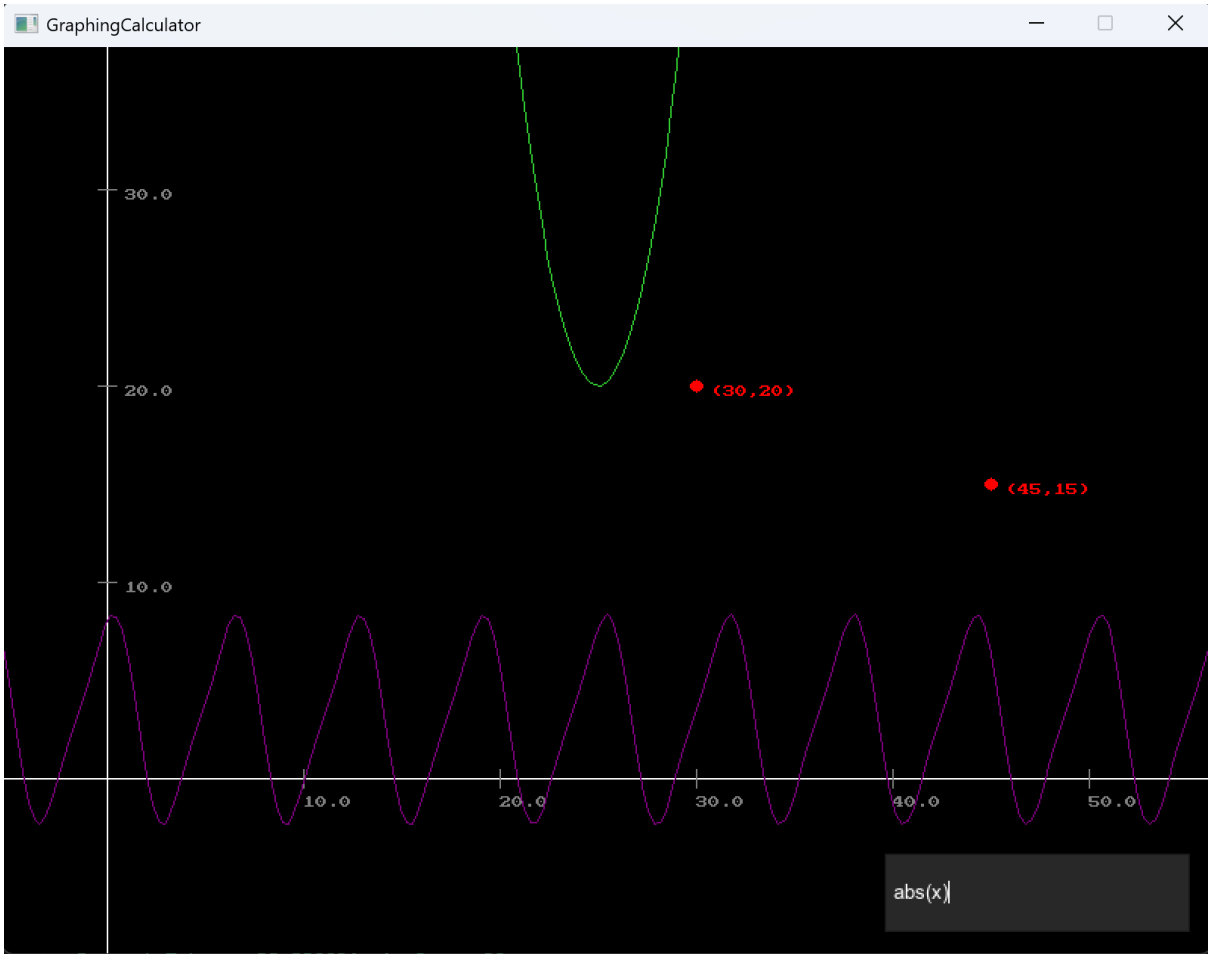
Design Overview for Graphing Calculator

Name: Ryan Weber
Student ID: 105914892

Summary of Program

Overall Summary

The program I have developed is a mathematical graphing calculator. Users can input graphs or coordinates in a window, and view them displayed on a cartesian plane. Users can pan and zoom around the plane to view the graphs.



The graphing calculator parses strings into graphs that it can display, and supports addition, subtraction, negative numbers, multiplication, division, exponents, various functions such as square root, various constants such as pi, brackets, and the interaction between all of these components. This make it flexible in showing a wide range of relations. The graphing calculator also displays points as dots on the cartesian plane, with their coordinates.

```
operations["sin"] = new SinOperation();
operations["cos"] = new CosOperation();
operations["tan"] = new TanOperation();
operations["log"] = new LogOperation();
operations["sqrt"] = new SqrtOperation();
operations["abs"] = new AbsOperation();
operations["power"] = new PowerOperation();
operations["multiply"] = new MultiplyOperation();
operations["star"] = new StarOperation();
operations["divide"] = new DivideOperation();
operations["add"] = new AddOperation();

constants["pi"] = MathF.PI;
```

Operations

The graphing calculator reloads graph calculations based on the transformation of the camera. It generates “nodes” based on the where the user camera is, when changing position or scale. This means minimal processing is spent on calculating points on the graph, and that the detail of the graph changes based on how it is viewed. It also includes an X and Y axis, with values along markings shown on each axis. These could be expanded into grid lines.

```
1 reference
public void AssessReload(float _reloadDistThreshhold, float _reloadSizeThreshhold)
{
    //distance, based on size as well
    if (MathF.Sqrt(
    (float)MathF.Pow((reloadPos.X - cam.CamPos.X) / cam.CamSize, 2) +
    (float)MathF.Pow((reloadPos.Y - cam.CamPos.Y) / cam.CamSize, 2)
    ) >= _reloadDistThreshhold ||
    cam.CamSize >= reloadSize * (1 + _reloadSizeThreshhold) ||
    cam.CamSize <= reloadSize * (1 - _reloadSizeThreshhold))
    {
        Reload();
    }
}

1 reference
public void Reload()
{
    //loop through and run draw object on every object
    foreach (GraphObject graphObject in graphObjects)
    {
        graphObject.Reload();
    }

    reloadPos = new Vector2(cam.CamPos.X, cam.CamPos.Y);
    reloadSize = cam.CamSize;
}
```

Reloading when camera moves or changes size

How this is program useful

Graphing calculators are used to visually represent mathematical functions, in order to gain a deep understanding of how any functions work, and how equations are connected to outputs. Furthermore, they can be used to solve mathematical problems. Graphing calculators are commonly used in education by students in schools, for help in teaching parts of maths, however, they are also useful for visualising functions in a wide range of circumstances in the real world.

How to Use the Program

Use dotnet run to run the graphing calculator.

Graph Inputs

The main function can take in graphs as arguments.

```
dotnet run "0.5x^(sin(2x))"
```

Use the on-screen text box to type in a graph or coordinate. Use the [enter] key to add it to the cartesian plane. Graph entries should be in expression format. Do not include “y=” or alike. Examples are shown below.

Input text box

The program supports many types of mathematical functions and expressions. Spaces are removed from inputs, and do not signify anything. For example, “5 * 5+5” will equate to 30. Traditional BODMAS Rules apply. Functions are calculated before multiplication, meaning inputs such as “sin2x” will calculate as “sin(2)*x”. Operators such as “+”, “^” and “/” will look for numbers on either side of them.

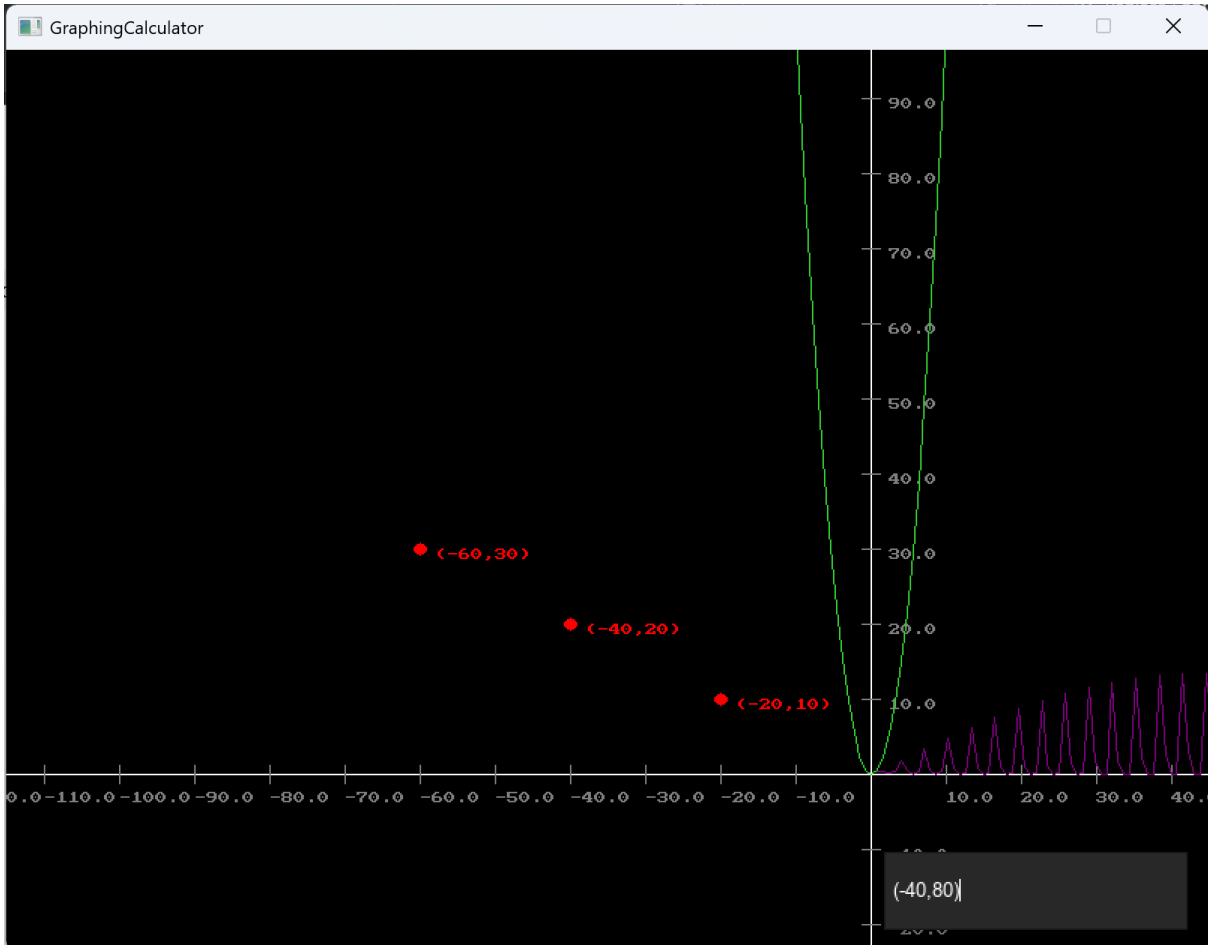
```
//find operation and simplify
foreach (KeyValuePair<string, IOperation> operation in operations)//for each operation type
{
    string operationName = operation.Key;//store name
    if (operations[operationName].FitsOperation(_tokens, i) && operations[operationName].Priority < lowestPriority)//if lower
    {
        lowestPriority = operations[operationName].Priority;//reset lowest
        lowestPriorityActionKey = operationName;//new action
        indexToModify = i;//new index to modify
    }
}
```

BOSMAS searching

Many forms of input should be accepted by the calculator. If any input is not in a correct format, it will output an error message in the console. Despite this, some inputs may not technically have errors, but may not appear as intended due to poor formatting.

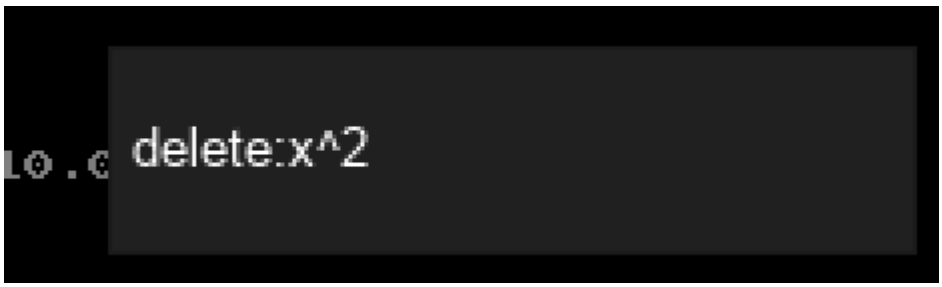
Invalid entry warning

Coordinates can be inputted in the form “(x,y)”



Entering coordinates

Writing “delete:” before a graph or coordinate will remove all copies of it from the cartesian plane.



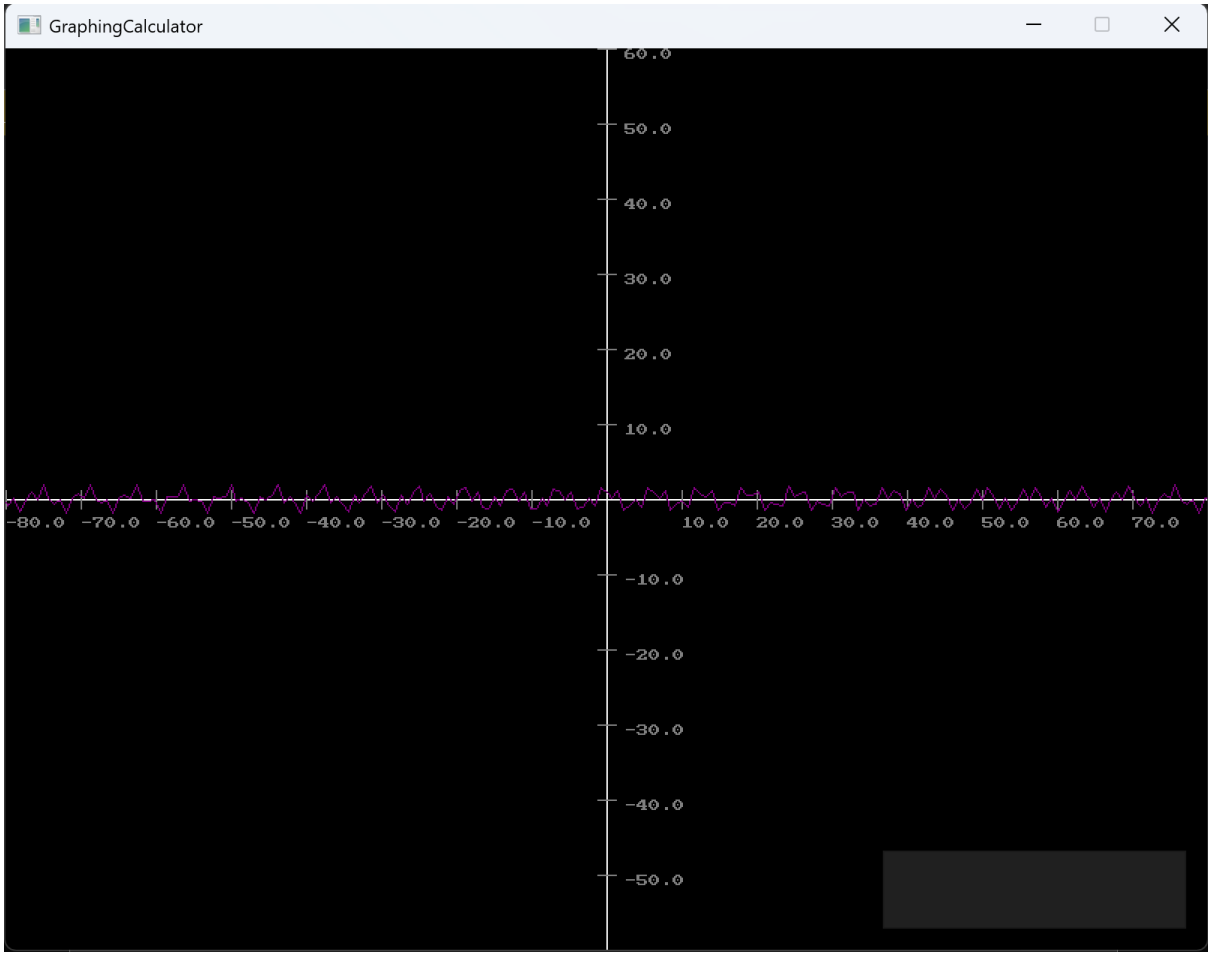
Input Formatting Examples

Here are some examples of various valid inputs. Note that these only show structure, and various functions can be used:

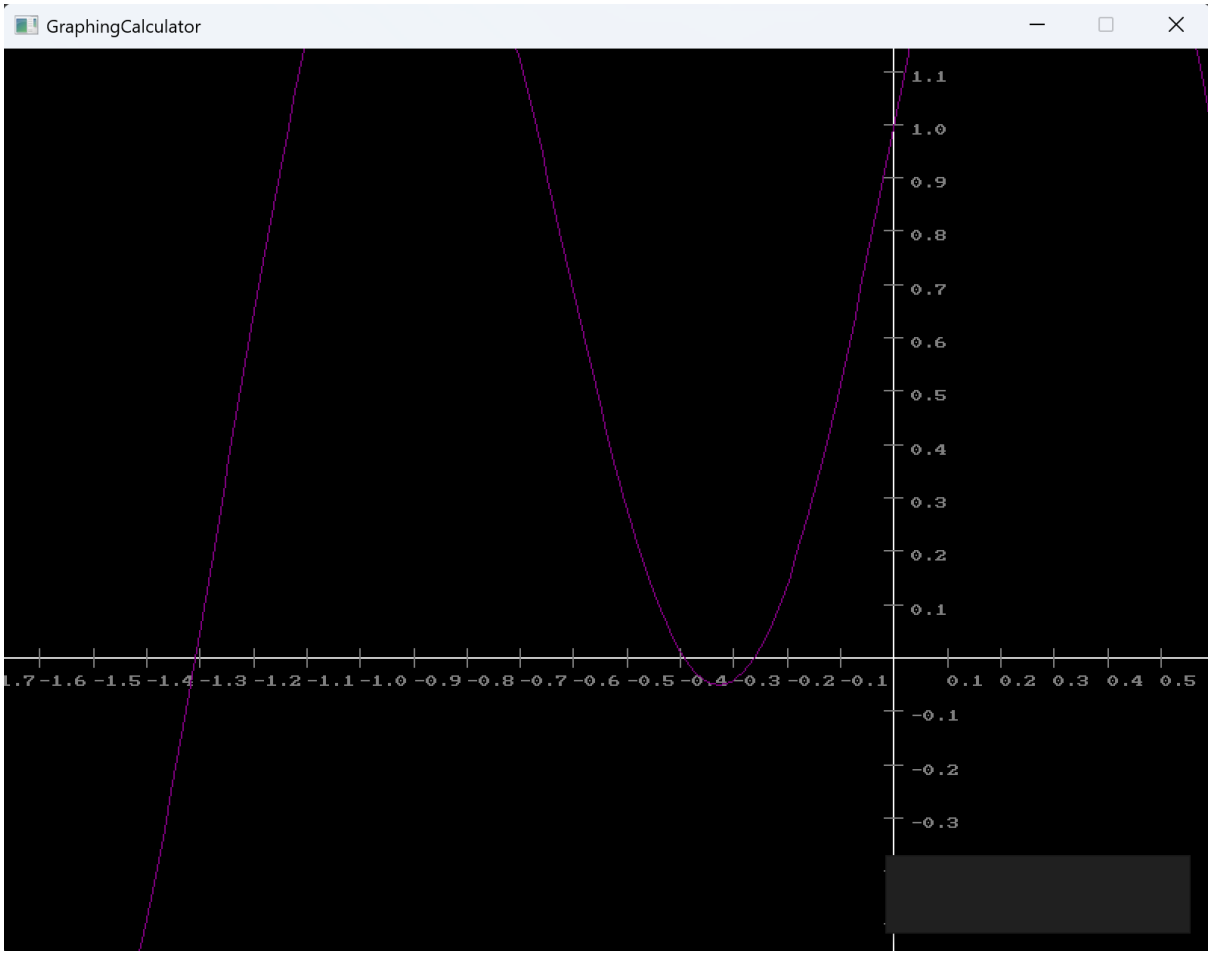
- 2x
- -2x
- 0.2x
- 2(x)
- x^2

- x^-0.2
- x+2
- x+x
- (x)(x)
- x/x
- sin(x)
- sinx
- sin(x)cos(x)
- sin(pi)
- sinxpicos2xe
- -x-2
- -2x-2
- 2-x
- 2/x
- e^x
- (-5,20.5)
- delete:2x
- 0.01x^3-0.5x^2+10sin(0.2x)+5cos(0.1x)+54.02

Camera Controls
Use the [W][A][S][D] keys to move the camera, and the [E] and [Q] keys to zoom in and out. Movement speed is based on the camera size.


GraphingCalculator
60.0
50.0
40.0
30.0
20.0
10.0
-80.0 -70.0 -60.0 -50.0 -40.0 -30.0 -20.0 -10.0
10.0 20.0 30.0 40.0 50.0 60.0 70.0
-10.0
-20.0
-30.0
-40.0
-50.0

Zooming out



Zooming in

How the Program Works

This is a brief overview of how the graphing calculator functions.

- Inputs are taken from the console, or the window, in string format
- Inputs are cleaned and determined whether they represent a graph, coordinate, or command (like delete)
- A GraphObject class is inherited by many classes which are represented on the cartesian plane. This includes the axis and markings.
- If a point (coordinate) is entered, it will create a Point object at specified coordinates.
- If a graph is entered:
    - Operations and multi-character components (such as pi or cos) will be stored in the object.
    - The position of a number of “nodes” will be calculated. The graph will draw between these nodes to show the graph.
    - For each node, the Y height for the node’s X value must be evaluated:
        - First (state 1, TokenGeneration()), the string is turned into an array of “tokens”:
            - Tokens can be different, such as a * symbol which only function is to separate numbers, or a function like cos.

- Numbers are built over iterations of characters.
- "-" can indicate a negative number, or an operation.
- X is replaced with the x value at that node.

▪ Second (stage 2 / TokenEvaluation()), the tokens are simplified:

- The token list is scanned, and the highest priority operation is tracked.
- If an end bracket is found (this indicates an inner-most bracket), the scan stops and the stage 2 function recursively runs on the contents of the brackets, and deletes them.
- If a bracket was not found, the highest priority operation is executed at the appropriate point. The logic is determined by the relevant IOperation class.
- This shortens the list. This second stage is repeated until there is only a single element in the list, which must be the Y value.

o Every frame, a line is drawn between every node in every graph, considering the camera's transform. The points at which a graph object is drawn is based off its "world" coordinates, as well as the transform of the camera.

o If there is a substantial change in the camera's transform, the nodes from all graphs will be calculated again. The points at which the nodes are calculated are based on the new transform of the camera.

```
if (_tokens[i].Word == ")")//brackets are special
{
    List<Token> insideBracketTokens = new List<Token>();

    _tokens.RemoveAt(i);//delete end bracket
    i--;
    while (_tokens[i].Word != "(")
    {
        insideBracketTokens.Insert(0, _tokens[i]);
        _tokens.RemoveAt(i);
        i--;
    }
    _tokens.RemoveAt(i);//delete start bracket

    _tokens.Insert(i, new Token(EvaluateTokens(insideBracketTokens), this));//recursive function!!! this is so cool!

    foundBracket = true;
    break;
}
```

Recursive Function

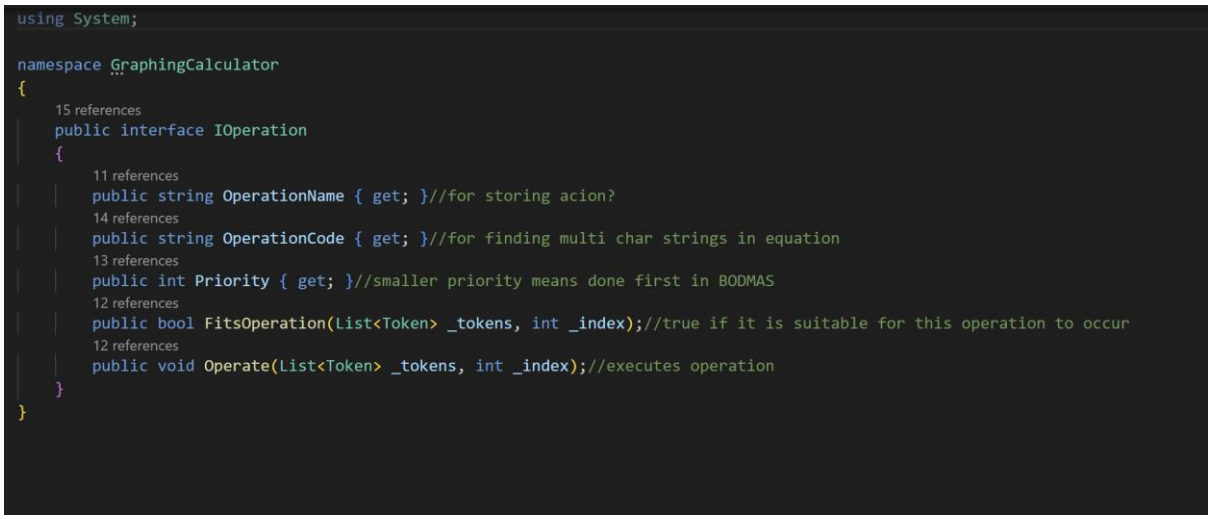
OOP Practices

Throughout the development of my program, I have implemented many OOP practices in order to assist future development, and maintain the flexibility and readability of my code.

Parsing

A big part of my work on this project has been focussed on parsing a user-given input into a function that can give a Y output given an X input. Originally I was concerned with how I could incorporate proper OOP practices into this, however while programming, I developed

ways to handle and control the growing complexity of the program, providing a higher cohesion. This was especially important as I was dealing with large, complicated functions that could be hard to understand without the proper practices.



```
using System;

namespace GraphingCalculator
{
    15 references
    public interface IOperation
    {
        11 references
        public string OperationName { get; }//for storing acion?
        14 references
        public string OperationCode { get; }//for finding multi char strings in equation
        13 references
        public int Priority { get; }//smaller priority means done first in BODMAS
        12 references
        public bool FitsOperation(List<Token> _tokens, int _index);//true if it is suitable for this operation to occur
        12 references
        public void Operate(List<Token> _tokens, int _index);//executes operation
    }
}
```

IOperation Interface

Operation Interface

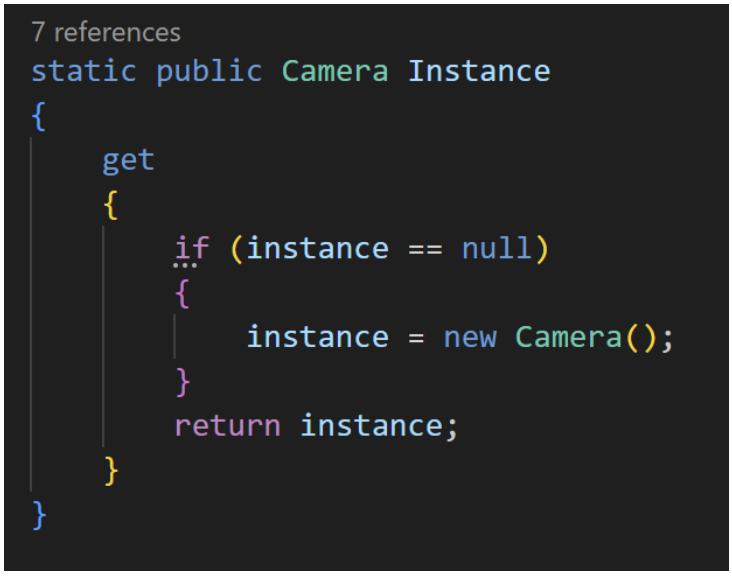
I mainly accomplished this through the IOperation interface.

Instead of using switch and case to determine which operation to use, I implemented an “IOperation” interface, which encapsulated properties for Name, Code, and BODMAS Priority, as well as methods for determining when a certain operation could be applied, and executing that operation. I used the interface in operation classes, which were added to a dictionary, which was iterated through my graphs.

At earlier stages, I was using multiple dictionaries and lists to determine operations, their priorities, and which components were multiple characters long. However, this was later merged into using the single Operation class. This enables future operations to be much easy to add and modify, and simplified the structure of the program, since the lower-level functions did not have to be edited. I used this to condense code in the GenerateTokens() and EvaluateTokens() classes.

Camera Singleton Design Pattern

I also used the Singleton design pattern for the camera, which was useful, since there were multiple classes that need to access it. In the beginning of development I was passing camera information through multiple functions, however as camera size was added, this became confusing.



Singleton

Other OOP Principles

Throughout the program, I used structure of inheritance and aggregation with objects on the graph, which can be seen in the UML structure diagram. Classes which aggregated their own objects such as Grid and Graph were used to control their rendering, as they had the most information about how they should behave.

In general, the program is built in a way that makes it easy to modify constant variables are used instead of numbers in most cases where they can be, and behaviours are separated by class, for example, graph input is left to the Input class, and Axis and Gridlines are also separated. This makes for loose coupling. I often implemented certain ways of OOP programming despite knowing it would not benefit me in the short term, but was a good practice.

Difficulties

Although I made separate classes for Render, Program, and Camera, the lines between them were sometimes blurry, and it was hard to determine which class should encapsulate a particular field or function. This posed as a challenge in developing this program.

Some parts of this program, although completed in the end, posed as a challenge, such as subtraction. When entering graphs like -x or x-5, the “-“ (subtraction) sign can be used in different ways, and have different meanings, making it difficult to determine and code. In general, the complicated logic in this program gave some headaches, but were made easier by using proper OOP principles.

What I Have Learned

Developing this program and completing this unit have taught me much about the principles of Object-Oriented Design. I have previously worked with c#, but I had minimal knowledge

of these principles and the way of thinking in sound OOP, which I was surprised to learn about.

If I had to make this program again, I would focus more on planning and coding interfaces and structure before adding content. This would make it easier and more efficient to develop a sound program. I spent quite some time refactoring code to be cleaner and work better for the future. I would also do more research on the generally accepted ways of coding different types of design patterns, as although I understood many design patterns, I did not have experience with implementing them in code.

If I could work more on the program, I would implement a way to stop asymptotes from rendering incorrectly, and I may adjust reloading so that lag spikes aren't as frequent.

The principles of OOP design is something I will continue to follow and improve on when programming in the future, as it is a hobby of mine. I would also like to investigate refining my own coding practices, to limit inconsistencies in my code. I often question the best way to implement a certain feature, so it would be effective to create my own coding standard that fits with OOP principles.

I have enjoyed learning and thinking about OOP. It has helped me a become a better programmer, and view not only code, but also other parts of the world, in a different way.

Roles

Table 1: Render – Holds graph objects and camera / rendering details

Table 2: AbsOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
prioroty	Private, int	
OperationName	Public, string, property	Overrides interface
Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, directly compares with string "abs"
Operate()	Public, void, method	Overrides interface, executes abs on value then removes function

Table 2: AddOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
Prioroty	Private, int	
OperationName	Public, string, property	Overrides interface

Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, directly compares with string “+”
Operate()	Public, void, method	Overrides interface, executes + on either side and deletes excess

Table 3: CosOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
Prioroty	Private, int	
OperationName	Public, string, property	Overrides interface
Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, directly compares with string “cos”
Operate()	Public, void, method	Overrides interface, executes cos on value and deletes excess

Table 4: DivideOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
Prioroty	Private, int	
OperationName	Public, string, property	Overrides interface
Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, directly compares with string “/”
Operate()	Public, void, method	Overrides interface, executes / on either side and deletes excess

Table 5: LogOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
Prioroty	Private, int	
OperationName	Public, string, property	Overrides interface
Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, directly compares with string “ln”

Operate()	Public, void, method	Overrides interface, executes natural log on value and deletes excess

Table 6: MultiplyOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
Prioroty	Private, int	
OperationName	Public, string, property	Overrides interface
Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, sees if there are 2 numbers in a row with nothing in between
Operate()	Public, void, method	Overrides interface, executes * on either side and deletes excess

Table 7: PowerOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
Prioroty	Private, int	
OperationName	Public, string, property	Overrides interface
Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, directly compares with string “^”
Operate()	Public, void, method	Overrides interface, executes power operation on either side t and deletes excess

Table 8: SinOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
Prioroty	Private, int	
OperationName	Public, string, property	Overrides interface
Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, directly compares with string “sin”

Operate()	Public, void, method	Overrides interface, executes sin on value and deletes excess

Table 9: SqrtOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
Prioroty	Private, int	
OperationName	Public, string, property	Overrides interface
Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, directly compares with string “sqrt”
Operate()	Public, void, method	Overrides interface, executes square root on value and deletes excess

Table 30: StarOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
Prioroty	Private, int	
OperationName	Public, string, property	Overrides interface
Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, directly compares with string “*”
Operate()	Public, void, method	Overrides interface, Just deletes itself, because multiplication is handled by MultiplyOperation

Table 41: TanOperation : IOperation

Responsibility	Type Details	Notes
operationName	Private, string	
Prioroty	Private, int	
OperationName	Public, string, property	Overrides interface
Priority	Public, int, property	Overrides interface
OperationCode	Public, string, property	Overrides interface
FitsOperation()	Public, bool, function	Overrides interface, directly compares with string “tan”
Operate()	Public, void, method	Overrides interface, executes tan on value and deletes excess

Table 52: Render

Responsibility	Type Details	Notes
cam	Private, Camera	References camera instance
graphObjects	Private, GraphObject List	Stores all graph objects which are to be rendered. Anything graph object wanting to draw to the screen must be in this list. Modifiable at runtime.
reloadPos	Private, Vector2	Last graph reload spot
reloadSize	Private, float	Last size when graph reloaded
Render()	Public, Constructor	Makes a new Render with a grid (markings) and axis lines
DrawFrame()	Public, void, method	Loops through every graph object in the list and calls DrawObject()
AddGraphObject()	Public, void, method, in _input : string, _colour : Color	Adds GraphObjects to the plane.
DeleteGraphObject()	Public, void, method, in _input : string	Deletes graphObjects with this equation
AssessReload()	Public, void, method, in _reloadDistThreshhold : float, _reloadSizeThreshhold : float	Calls Reload() If the camera transform has changed too much
Reload()	Public, void, method	Calls Reload() on graphObjects and resets reloadPos and reloadSize

Table 12: GraphObject – Objects to be drawn to the screen on the cartesian plane. Is a parent class.

Responsibility	Type Details	Notes
position	protected, Vector2	X Y position of this GraphObject. Not needed for graphs
colour	protected, Color	Colour used to display this graph object
cam	Protected, Camera	Camera
equation	Protected, string	This was initially only in graph, but is used by point to determine deletion

GraphObject()	Public, Constructor	Makes a new graph object with a colour of red and a (0, 0) position
GraphObject()	Public, Constructor, in _position : Vector2	Makes a new graph object with a colour of red and a given position
GraphObject()	Public, Constructor, in _ position : Vector2, in _colour : Color	Makes a new graph object with a given colour and a given position
Position	Public, Vector2, Property	Read and write property to position
Colour	Public, Color, Property	Read and write property to colour
Equation	Public, string, property	Read equation
Camera	Public, camera, property	Read camera
DrawObject()	Public, Virtual, void, method, in _camPos : Vector2	Is called by Render. Performs the appropriate draw action for that object. If it is not overridden, draws a dot at its position
Reload()	Public, virtual, void, method	Performs action to happen when reloaded
WorldToScreenX()	Public, float, function, in _x : float	Returns screen x of cartesian x
WorldToScreenY()	Public, float, function, in _y : float	Returns screen y of cartesian x

Table 13: Graph : GraphObject – Graph object, connected by nodes and drawn with lines. This is a big one…

Responsibility	Type Details	Notes
NODES_PER_GRAPH	Private, int	Nodes per graph
EDGE_BUFFER	Private, float	Portion that things are drawn outside of view. Unused currently, since camera one is used
nodes	Private, Node List	Stores nodes related to that graph, linking them together. This is changed at runtime.
posNumbers	Private, Char[]	Positive ints used to determine number build
multiCharComponents	Private, string list	Assigned dynamically
operations	Private, Dictionary<string, IOperation>	Possible operations
constants	Private, Dictionary<string, float>	Constant values like pi

Graph()	Public, Constructor, in _equation : string	Makes a new graph, creating node and calling Reload().
Graph()	Public, Constructor, in _equation : string, _colour, Color	With colour
DrawObject()	Public, Override, void, method	Draws lines between nodes, rendering the graph. (dot to dot!)
CalculateNodes()	Public, void, method, in _ equation : string, _nodeCount : int, _from : float, _to : float	Parent function for calculating every relevant node. Ran by reload. Resource intensive!
AddNode()	Public, void, method, in _ nodePos : Vector2	Adds node
EvaluateFunction()	Public, float, function, in _equation : string, _x : float	Parent function for Generate and Evaluate. Run for every x value
Reload()	Public, override, void, method	Runs CalculateNodes()
GenerateTokens()	Public, token list, function, in _equation : string, _x : float	Converts string to token list. Complicated. (stage 1)
EvaluateTokens()	Public, float, function, in _tokens : token list	Simplifies token list. Complicated. (stage 2)
DebugTokens()	Public, string, function, in _comps : token list	Outputs tokens. Very useful for testing

Table 14: Point : GraphObject – Single displayed coordinate. This class does not need a position or other data as it is inherited from GraphObject.

Responsibility	Type Details	Notes
Point()	Public, Constructor, in _equation : string, _x : float, _y : float	Creates a point
DrawObject()	Public, override, void, method	Calls base to draw point, but also draws text

Table 15: Node – Part of a graph used to link it together.

Responsibility	Type Details	Notes
noValue	Private, bool	Should I draw this? (could be NaN)
Node()	Public, Constructor, in _x : float, _y : float	Creates a node, determines noValue
NoValue	Public, bool, Property	Read noValue

Table 16: Vector2 – Custom class for storing 2D coordinates.

Responsibility	Type Details	Notes
x	Private, float	X value of the coordinate

y	Private, float	Y value of the coordinate
Vector2()	Public, Constructor	Creates Vector2
Vector2()	Public, Constructor, in _point : Point2D	Creates Vector2 at an inputted Point2D. Didn't use this
Vector2()	Public, Constructor, in _x : float, in _y : float	Creates Vector2 at an inputted x and y coordinate
X	Public, float, Property	Read and write property to x
Y	Public, float, Property	Read and write property to y
+()	Public, Static, Operator, in _a : Vector2, in _b : Vector2	Adds together 2 Vector2s
***()**	Public, Static, Operator, in _a : Vector2, in _b : Vector2	Adds together 2 Vector2s
***()**	Public, Static, Operator, in _a : float, in _b : Vector2	Adds together Vector2 with float

Table 17: Camera (singleton)

Responsibility	Type Details	Notes
instance	Private, static, Camera	Private instance
camPos	Private, Vector2	Cam coords
camSize	Private, float	Cam size
SCREEN_RES_X	Private, int	Screen res x
SCREEN_RES_Y	Private, int	Screen res y
EDGE_BUFFER	Private, float	How far out of the screen things are drawn
CamPos	Public, Vector2, property	Get and set
CamSize	Public, float, property	Get and set
ScreenResolutionX	Public, int, property	Get and set
ScreenResolutionY	Public, int, propery	Get and set
CamBoundPosX()	Public, float	World position of boundary, used for range calculations
CamBoundPosX()	Public, function, float, in _doEdgeBuffer : bool	World position of boundary, used for range calculations. Do we consider buffer?
CamBoundNegX()	Public, function, float	World position of boundary, used for range calculations
CamBoundNegX()	Public, function, float, in _doEdgeBuffer : bool	World position of boundary, used for range calculations. Do we consider buffer?

CamBoundPosY()	Public, function, float	World position of boundary, used for range calculations
CamBoundPosY()	Public, function, float, in _doEdgeBuffer : bool	World position of boundary, used for range calculations. Do we consider buffer?
CamBoundNegY()	Public, function, float	World position of boundary, used for range calculations
CamBoundNegY()	Public, function, float, in _doEdgeBuffer : bool	World position of boundary, used for range calculations. Do we consider buffer?
Instance	Static, public, property, Camera	Get. Singleton, makes new class if there isn't one already.

Table 18: AxisLine : GraphObject

Responsibility	Type Details	Notes
direction	Private, int	X or y. for enum
Direction	Private, Enum	X and y
AxisLine()	Public, Constructor, in _direction : int	Constructor
AxisLine()	Public, Constructor, in _direction : int, _colour : Color	Constructor with colour
DrawObject()	Public, override, void, method	Draws massive line using cam details

Table 19: Grid : GraphObject

Responsibility	Type Details	Notes
gridLines	Private, GridLine list	Stores lines, like how graph stores nodes
GRIDLINE_SCALING_FACTOR_HORIZONTAL	Private, float	How often gridlines convert between powers of 10 horizontally
GRIDLINE_SCALING_FACTOR_VERTICAL	Private, float	How often gridlines convert between powers of 10 vertically
Grid()	Public, Constructor, in _colour : Color	Reload()s
DrawObject()	Public, override, void, method	Draws all gridLines

Reload()	Public, override, void, method	Recalculates lines

Table 20: GridLine : GraphObject

Responsibility	Type Details	Notes
direction	Private, int	Horizontal or vertical
Direction	Private, Enum	X, y
LINE_LENGTH	Private, int	Const line length
GridLine()	Public, constructor, in _position : Vector2, _dircection : int, _colour : Color	Constructor
DrawObject()	Public, override, void, method	Draws based on direction

Table 21: Input – text input

Responsibility	Type Details	Notes
cam	Private, Camera	Cam
Rectangle	Private, Rectangle	Needed for textbox
inputText	Private, string	Can't be temporary
Input()	Public, constructor	constructor
GetInput()	Public, string, function	Splashkit input through textbox

Table 22: IOperaiton

Responsibility	Type Details	Notes
OperationName	Public, string, property, get	Name for key in BODMAS
OperationCode	Public, string, property, get	Used for length, could also be used for FitsOperation()
Priority	Public, int, property, get	For BODMAS
FitsOperation()	Public, bool, function, in _tokens : token List, _index : int	Determines if token references it
Operate()	Public, void, method, in _tokens : token list, _index : int	Executes operation

Table 23: Token

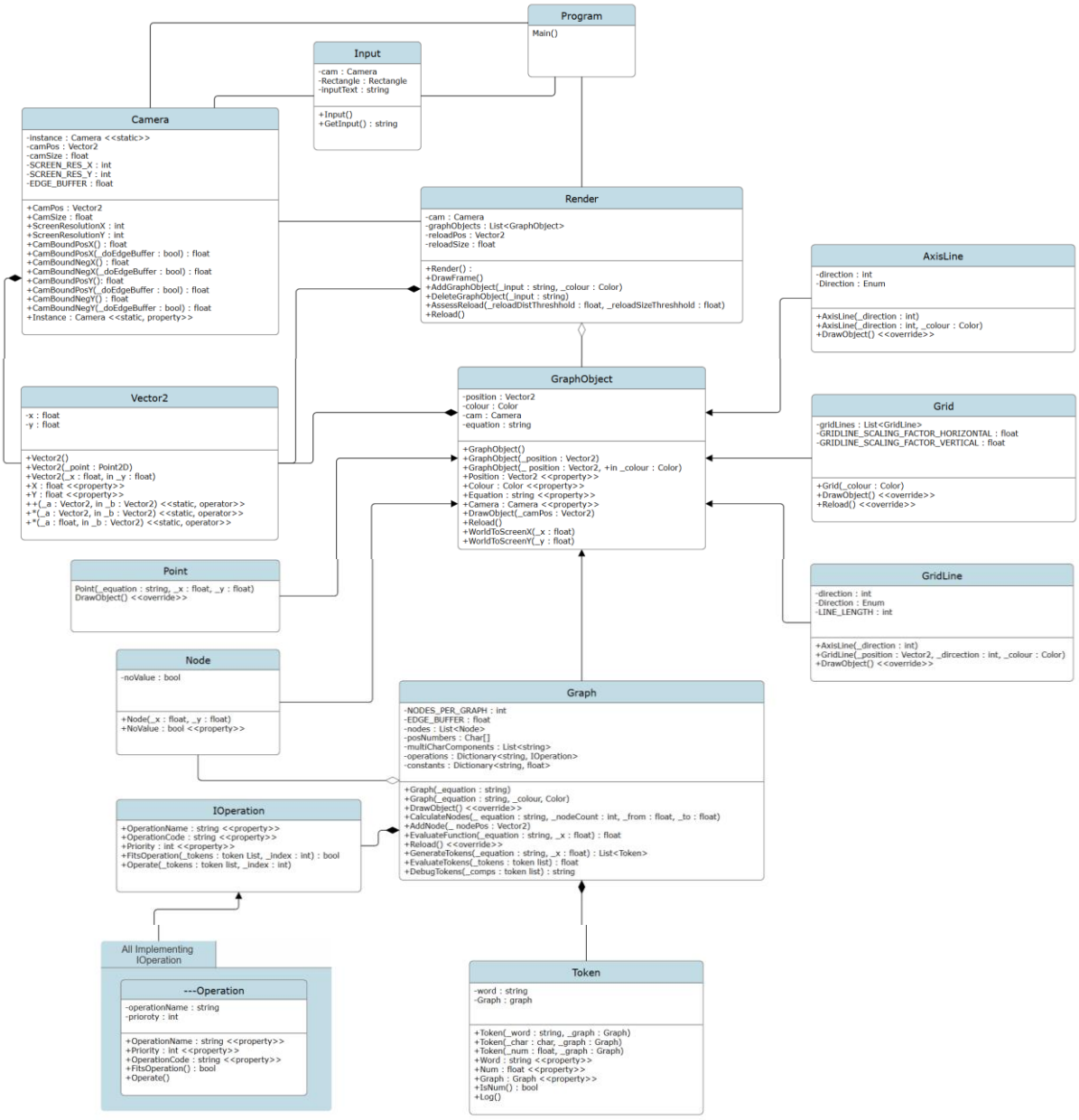
Responsibility	Type Details	Notes
word	private, string	What string is stored in this token? This is the main field

Graph	Private, graph	Unused. Used to implement FlyWeight by referencing list, now we use Parse.
Token()	Public, constructor, in _word : string, _graph : Graph	Creates token from variable
Token()	Public, constructor, in _char : char, _graph : Graph	Creates token from variable
Token()	Public, constructor, in _num : float, _graph : Graph	Creates token from variable
Word	Public, string, property	Get set
Num	Public, float, property	Get set, and parses!
Graph	Public, Graph, property	Get
IsNum()	Public, bool, function	Determines if this token represents a number
Log()	Public, void, method	Writes the word of the token. Useful for testing

Table 23: Program - and fields in Main

Responsibility	Type Details	Notes
cam	cam	cam
input	Input	input
CAM_MOVE_SPEED	Float	Speed that cam moves, controlled by program
CAM_ZOOM_SPEED	Float	Scaling rate proportional to size already
timer	Stopwatch	For deltaTime
prevTime	Float	For deltaTime
RELOAD_DISTANCE_THRESHHOLD	Float	Reload value
RELOAD_SIZE_THRESHHOLD	Float	Reload value
window	Window	window
render	Render	render
BACKGROUND_COLOUR	Color	Bg colour
Main()	Public, static, void, method, in _args : string[]	Does setup and loop. Everything comes from this

Class Diagram



Sequence Diagram